AI Methodology Appendix

P.R.I.M.E. Documentation of AI Co-Pilot Usage

ECON 3916: ML Prediction Project | Spring 2026

Overview

Throughout this project, I used Claude (Anthropic) as an AI co-pilot for code generation, debugging, and structuring the analysis pipeline. Every AI-generated output was verified by running the code, checking results against domain knowledge, and iterating on errors. Below are three representative prompt-response interactions documented using the P.R.I.M.E. framework: Prep, Request, Iterate, Mechanism Check, Evaluate.

AI tools were used for: (1) data acquisition and cleaning pipeline, (2) model training and hyperparameter tuning code, and (3) Streamlit dashboard development. All outputs were tested end-to-end in Google Colab and locally before deployment.

Interaction 1: Data Pipeline and All-Star Labeling

P - Prep

I needed to load NBA per-game statistics from a Kaggle dataset and merge with All-Star selection records to create a binary classification target. I had already identified the dataset (Sumitro Datta, NBA/ABA/BAA Stats) and knew the two relevant files: 'Player Per Game.csv' and 'All-Star Selections.csv'. Key challenges: handling traded players who appear in multiple rows, filtering to relevant seasons, and matching player names across the two files.

R - Request

Prompt: 'I have a Kaggle dataset with Player Per Game.csv and All-Star Selections.csv. The season column uses integer years (e.g., 2024 for 2023-24). I need to: filter to 2015-2024, handle traded players by keeping the TOT row, require minimum 20 games, and create an all_star binary label by merging with the All-Star file. Give me the full cleaning and merging code.'

I - Iterate

The first version of the code attempted to scrape Basketball Reference directly, which failed due to 403 errors from Colab. I iterated by switching to the Kaggle CSV approach. A second iteration was needed when the initial All-Star labeling used asterisks in player names (Basketball Reference format), which were absent in the Kaggle data. The final version used a set-based merge on (player, season) tuples from the All-Star Selections file, which worked correctly.

M - Mechanism Check

Verification steps: (1) Printed the top 15 scorers in 2023-24 and confirmed known All-Stars (LeBron James, Stephen Curry, Nikola Jokic) were labeled correctly. (2) Checked All-Star counts per season (expected ~24 per year, observed consistent counts). (3) Confirmed the overall positive rate (~6.3%) matched expectations (~240 All-Stars across 4,255 player-seasons). (4) Verified no duplicate player-season rows remained after the groupby operation.

E - Evaluate

The AI-generated pipeline required three iterations to handle real-world data issues (403 blocks, missing asterisks, column name mismatches). Each failure was diagnosed by printing intermediate outputs. The final pipeline is correct and reproducible. My judgment was required to: choose the minimum-games threshold (20), decide to use the TOT row for traded players, and confirm the All-Star labeling via sanity checks.

Interaction 2: Model Training and Hyperparameter Tuning

P - Prep

I needed to compare three classification models (Logistic Regression, Random Forest, Gradient Boosting) with proper cross-validation, hyperparameter tuning, and handling of the ~94:6 class imbalance. I knew from the course material (Ch 6, Ch 15) that I needed stratified K-fold CV, GridSearchCV, and class weighting. I wanted F1 as the primary metric since accuracy is misleading with imbalanced classes.

R - Request

Prompt: 'Build training code for 3 models: Logistic Regression (with StandardScaler and class_weight balanced), Random Forest (class_weight balanced), and Gradient Boosting. Use 5-fold stratified CV, GridSearchCV for hyperparameter tuning, and evaluate on a held-out test set with bootstrap 95% confidence intervals. Features are 13 per-game stats, target is all_star, random_state=42 everywhere.'

I - Iterate

The initial Gradient Boosting grid had 32 parameter combinations and was slow on Colab. I reduced the grid to 4 combinations (fixing learning_rate=0.1 and min_samples_leaf=5) for practical runtime. I also combined the test evaluation, bootstrap CI, and plotting cells into a single cell to avoid NameError issues from variables not persisting across Colab cell executions.

M - Mechanism Check

Verification: (1) Confirmed train/test split preserved the class ratio (~6.3% in both sets). (2) Checked that Random Forest and Gradient Boosting were trained on unscaled data while Logistic Regression used scaled data. (3) Verified confidence intervals were reasonable (not degenerate). (4) Compared AUC-ROC values across models to confirm all three significantly outperform random (0.5). (5) Checked that the best F1 model (Random Forest) matched my expectation based on CV results.

E - Evaluate

The AI correctly structured the modeling pipeline with proper data leakage prevention (scaler fit on training data only, CV within training set only). My judgment calls: (1) choosing F1 over accuracy as the selection metric, since accuracy would favor always predicting 'not All-Star'; (2) using class_weight='balanced' rather than SMOTE, since the course material emphasized this approach; (3) reducing the GB grid for practical runtime while keeping the most impactful hyperparameters.

Interaction 3: Streamlit Dashboard Development

P - Prep

I needed to build a Streamlit app with two interaction modes: (1) a player lookup where users select a current-season player and see their All-Star probability, and (2) a custom stats mode with sliders for all 13 features. The app needed to load the saved model artifacts (pickle file), display predictions with uncertainty, and include at least one interactive visualization. I saved model_artifacts.pkl, nba_allstar_dataset.csv, and current_season_stats.csv from the Colab notebook.

R - Request

Prompt: 'Build a Streamlit app (app.py) for NBA All-Star prediction. It should have: (1) a player lookup tab with a dropdown, stats display, and a Plotly gauge chart for probability; (2) a custom stats tab with sliders and a bar chart comparing user stats to All-Star averages; (3) a model comparison tab showing the performance table and feature importance. Load models from model_artifacts.pkl. Include the predictive-not-causal caveat. Also create requirements.txt with pinned versions.'

I - Iterate

The initial version needed adjustment for the scikit-learn version pin (1.6.1 to match Colab's version, since pickle files are version-sensitive). I also refined the sidebar to show model performance metrics and added a model selector dropdown so users could compare predictions across all three models.

M - Mechanism Check

Verification: (1) Ran 'streamlit run app.py' locally and confirmed all three tabs rendered correctly. (2) Selected known All-Stars (LeBron James, Stephen Curry) and confirmed high predicted probabilities. (3) Selected a low-minutes bench player and confirmed low probability. (4) Tested edge cases on sliders (minimum and maximum values). (5) Verified that switching between models updated the prediction output. (6) Confirmed the Logistic Regression path correctly applied the scaler while RF and GB did not.

E - Evaluate

The AI produced a functional, well-structured app that required only minor version pinning adjustments. My judgment was applied to: (1) the UX design decisions (tabs vs. sidebar, gauge chart vs. simple number); (2) ensuring the 'predictive, not causal' caveat appeared prominently in the sidebar and model comparison tab; (3) choosing Plotly for interactive charts over static matplotlib; and (4) selecting the default slider values to represent a typical starting-caliber player.

Summary of AI Usage

Project Phase	AI Tool	Usage	Verification
Data acquisition	Claude	Scraping code, CSV loading, merging logic	Sanity checks on known All-Stars
EDA	Claude	Visualization code (histograms, boxplots, heatmaps)	Visually inspected all plots
Modeling	Claude	GridSearchCV, cross-validation, bootstrap CIs	Checked metrics, class ratios, leakage
Dashboard	Claude	Streamlit app with Plotly charts	Local testing, edge case inputs
Report	Claude	LaTeX/PDF structure and prose drafting	Edited for accuracy, added judgment

In all cases, AI-generated code was tested by running it end-to-end, and AI-generated prose was reviewed for accuracy. No AI output was used without verification. The primary value of the AI co-pilot was accelerating code generation and debugging, while domain judgment (feature selection, model evaluation, stakeholder framing) remained my responsibility throughout.